Spécifications Définitives : Système d'Annotations UML

Bibliothèque de Génération Java vers PlantUML

Documentation Architecturale

26 juin 2026

Table des matières

1 Introduction **3**

2 Le Déclencheur et le Seuil : `@UMLClass` **3**
- 2.1 Principe . . . 3
- 2.2 Le paramètre de Stratégie (Visibilité) . . . 3
- 2.3 Le Filtre Anti-Bruit Intelligent (Smart Default) . . . 3
- 2.4 Pourquoi c'est utile ? . . . 3
- 2.5 Paramètres . . . 4
- 2.6 Exemple . . . 4

3 La Dérogation d'Exclusion : `@UMLExclude` **4**
- 3.1 Principe . . . 4
- 3.2 Pourquoi c'est utile ? . . . 4
- 3.3 Paramètres . . . 4
- 3.4 Exemple . . . 4

4 La Dérogation d'Inclusion : `@UMLInclude` **5**
- 4.1 Principe . . . 5
- 4.2 Pourquoi c'est utile ? . . . 5
- 4.3 Paramètres . . . 5
- 4.4 Exemple . . . 5

5 La Précision Métier : `@UMLAssociation` **5**
- 5.1 Principe . . . 5
- 5.2 Pourquoi c'est utile ? . . . 5
- 5.3 Paramètres . . . 5
- 5.4 Exemple . . . 6

6 La Classe d'Association : `@UMLAssociationClass` **6**
- 6.1 Principe . . . 6
- 6.2 Pourquoi c'est utile ? . . . 6
- 6.3 Paramètres . . . 6
- 6.4 Exemple . . . 6

7 Les Éléments Dérivés : `@UMLDerived` 7
7.1 Principe . . . 7
7.2 Pourquoi c'est utile ? . . . 7
7.3 Paramètres . . . 7
7.4 Exemple . . . 7

8 Les Contraintes Métier : `@UMLConstraint` 7
8.1 Principe . . . 7
8.2 Pourquoi c'est utile ? . . . 7
8.3 Paramètres . . . 7
8.4 Exemple . . . 8

9 Les Liens Transversaux : `@UMLDependency` 8
9.1 Principe . . . 8
9.2 Pourquoi c'est utile ? . . . 8
9.3 Paramètres . . . 8
9.4 Exemple . . . 8

10 La Documentation Visuelle : `@UMLNote` 8
10.1 Principe . . . 8
10.2 Pourquoi c'est utile ? . . . 8
10.3 Paramètres . . . 9
10.4 Exemple . . . 9

11 Le Regroupement Logique : `@UMLPackage` 9
11.1 Principe . . . 9
11.2 Pourquoi c'est utile ? . . . 9
11.3 Paramètres . . . 9
11.4 Exemple . . . 9

12 Le Correcteur de Mise en Page : `@UMLLayout` 9
12.1 Principe . . . 9
12.2 Pourquoi c'est utile ? . . . 10
12.3 Paramètres . . . 10
12.4 Exemple . . . 10

1 Introduction

Ce document référence le système d'annotations Java conçu pour la génération automatique de diagrammes PlantUML. L'outil a été pensé pour allier la simplicité de la déduction automatique du code Java avec la flexibilité requise pour modéliser des concepts métier complexes et maîtriser le rendu visuel. Il gère nativement les concepts standards (héritage via `extends`, implémentation via `implements`, classes abstraites et énumérations) grâce à l'analyse de l'Arbre Syntaxique (AST). Les annotations ci-dessous viennent enrichir et paramétrer cette analyse.

2 Le Déclencheur et le Seuil : `@UMLClass`

2.1 Principe

Placée sur une classe, interface ou énumération, cette annotation indique que l'entité doit figurer sur le diagramme. Elle permet également de définir un seuil de visibilité par défaut pour ses attributs et méthodes, agissant comme un filtre d'inclusion.

2.2 Le paramètre de Stratégie (Visibilité)

Le paramètre `strategy` définit exactement quels niveaux d'encapsulation doivent être affichés automatiquement :

- **Aucune stratégie (par défaut)** : La classe apparaît sur le diagramme comme une coquille vide (aucun attribut ni méthode n'est comptabilisé).
- `PUBLIC` : Seuls les éléments `public` sont générés.
- `PROTECTED` : Les éléments `public` et `protected` sont générés.
- `PACKAGE` : Les éléments `public` et ceux sans modificateur (package-private) sont générés. Les éléments `protected` sont exclus par défaut.
- `PRIVATE` : Tous les éléments de la classe sont générés, sans aucune restriction.

2.3 Le Filtre Anti-Bruit Intelligent (Smart Default)

Pour éviter la surcharge visuelle du diagramme (le "bruit"), les méthodes bénéficient d'un traitement supplémentaire. Même si une méthode respecte le niveau de visibilité exigé par la `strategy`, le générateur ignore automatiquement :

- Les accesseurs et mutateurs (`getters`, `setters`, `is...`).
- Les constructeurs.
- Les méthodes natives héritées de `Object` (ex : `toString()`, `hashCode()`).

Seules les véritables méthodes "métier" sont générées par défaut. Il reste possible de forcer l'affichage d'un élément masqué avec `@UMLInclude`, ou de cacher une méthode métier spécifique avec `@UMLExclude`.

2.4 Pourquoi c'est utile ?

Sans cette annotation, le générateur tenterait de dessiner toutes les classes du projet, y compris les classes utilitaires ou les dépendances externes. Elle permet au développeur de choisir exactement le périmètre de son architecture, tout en garantissant un rendu final lisible et centré sur la valeur ajoutée grâce au filtre intelligent.

2.5 Paramètres

- `name` (String) : Surcharge le nom affiché (utile pour raccourcir les noms à rallonge).
- `color` (String) : Applique une couleur de fond.
- `stereotype` (String) : Ajoute une étiquette métier (ex : `"«Service»"`).
- `strategy` (Enum) : `PUBLIC`, `PROTECTED`, `PACKAGE`, `PRIVATE`.

2.6 Exemple

```
// Affiche automatiquement les elements autorises, mais filtre le bruit
@UMLClass(stereotype = "Entity", strategy = UMLVisibility.PACKAGE)
public class Utilisateur {
    public String pseudo;       // Sera affiche
    String email;               // Sera affiche (package-private)
    protected int age;          // Sera ignore (hors strategie)
    private String motDePasse;  // Sera ignore (hors strategie)

    public Utilisateur() {}     // Sera ignore (Filtre anti-bruit : constructeur)
    public String getPseudo() {}// Sera ignore (Filtre anti-bruit : getter)

    public void valider() {}    // Sera affiche (methode metier publique)
}
```

3 La Dérogation d'Exclusion : `@UMLExclude`

3.1 Principe

Cette annotation est une exception négative. Elle force la dissimulation d'un attribut ou d'une méthode, même si la `strategy` de la classe l'autorisait à apparaître.

3.2 Pourquoi c'est utile ?

Dans le cas où une classe possède une `strategy = PRIVATE` (tout afficher), certaines variables techniques (loggers, constantes internes) n'ont aucune valeur architecturale ou doivent être cachées. Les exclure permet d'alléger le diagramme final.

3.3 Paramètres

Aucun.

3.4 Exemple

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class ConnexionBase {
    private String url; // Affiche car strategie globale sur PRIVATE

    @UMLExclude
    private Logger log; // Ignore explicitement pour alleger le graphe
}
```

4 La Dérogation d'Inclusion : `@UMLInclude`

4.1 Principe

Cette annotation agit comme une exception positive à la règle de visibilité dictée par `@UMLClass`. Elle force l'apparition d'un attribut ou d'une méthode sur le diagramme, même si son niveau d'encapsulation aurait dû le masquer.

4.2 Pourquoi c'est utile ?

Si une classe n'a aucune `strategy` définie (classe vide par défaut), ou une `strategy = PUBLIC`, le développeur peut cibler très précisément les éléments internes (privés) cruciaux pour la compréhension de l'architecture sans avoir à exposer le reste.

4.3 Paramètres

Aucun.

4.4 Exemple

```
@UMLClass(strategy = UMLVisibility.PUBLIC)
public class MoteurDeRecherche {

    public void chercher() {} // Affiche car public

    private int compteur; // Ignore car private

    @UMLInclude
    private void algorithmeSecret() {} // Force l'affichage grace a l'
    annotation
}
```

5 La Précision Métier : `@UMLAssociation`

5.1 Principe

Surcharge la déduction automatique des relations portées par les attributs, en précisant la sémantique UML stricte, les cardinalités, les rôles et la classe ciblée.

5.2 Pourquoi c'est utile ?

Le code Java `List<Commande> commandes` indique qu'il y a plusieurs commandes. Mais seul le métier sait si c'est `0..*` ou `1..*`, et si c'est une simple association ou une composition forte. De plus, préciser explicitement la classe cible permet de pallier les limites de l'effacement de type (Type Erasure) sur les collections génériques.

5.3 Paramètres

— `target` (Class) : Pointe explicitement vers la classe cible de la relation.
— `type` (Enum) : `ASSOCIATION`, `COMPOSITION`, `AGGREGATION`.

— `sourceCardinality` (String) : Multiplicité côté source (ex : `"1"`, `"0..1"`).
— `targetCardinality` (String) : Multiplicité côté cible (ex : `"0..*"`, `"*"`).
— `sourceRole` et `targetRole` (String) : Noms des rôles affichés aux extrémités du lien.
— `isOrdered` (boolean) : Si vrai, ajoute la contrainte de collection `{ordered}`.
— `qualifier` (String) : Génère une association qualifiée (le petit carré contenant une clé d'identification).
— `label` (String) : Texte sur la flèche (ex : `"possede"`).
— `direction` (Enum) : `UP`, `DOWN`, `LEFT`, `RIGHT`.

5.4 Exemple

```
public class Utilisateur {
    @UMLAssociation(
        target = Commande.class,
        type = UMLRelationType.COMPOSITION,
        sourceCardinality = "1",
        targetCardinality = "0..*",
        targetRole = "achats",
        isOrdered = true,
        label = "passe"
    )
    private List<Commande> commandes;
}
```

6 La Classe d'Association : `@UMLAssociationClass`

6.1 Principe

Identifie une classe spécifique comme étant le porteur des attributs d'une relation entre deux autres classes (souvent utilisée pour modéliser des tables de jointure enrichies).

6.2 Pourquoi c'est utile ?

En Java, il est impossible de lier directement un objet à un lien. Il faut obligatoirement créer une classe intermédiaire. Cette annotation permet de signaler au générateur que cette classe intermédiaire doit être dessinée selon la norme UML stricte : une ligne en pointillés reliant le lien principal à la classe d'association.

6.3 Paramètres

— `source` (Class) : La classe de départ de la relation.
— `target` (Class) : La classe d'arrivée de la relation.

6.4 Exemple

```
@UMLAssociationClass(source = Etudiant.class, target = Cours.class)
@UMLClass
public class Inscription {
    private LocalDate dateInscription;
```

```
    private double noteObtenue;
}
```

7 Les Éléments Dérivés : `@UMLDerived`

7.1 Principe

Indique qu'une classe, un attribut ou une association est "dérivé", c'est-à-dire qu'il n'est pas instancié ou stocké directement, mais qu'il est calculé dynamiquement à partir d'autres données.

7.2 Pourquoi c'est utile ?

En UML, un élément dérivé doit être précédé d'un slash (`/`). Comme ce slash n'est pas une syntaxe Java valide pour nommer une variable, l'annotation signale au générateur d'ajouter ce préfixe visuel lors du rendu PlantUML.

7.3 Paramètres

Aucun.

7.4 Exemple

```
public class Personne {
    private LocalDate dateNaissance;

    @UMLDerived
    private int age; // Sera affiche en UML sous la forme : /age : int
}
```

8 Les Contraintes Métier : `@UMLConstraint`

8.1 Principe

Permet d'ajouter une règle formelle, exprimée entre accolades `{ }`, applicable à un objet, un attribut ou un lien.

8.2 Pourquoi c'est utile ?

Pour conserver la validité sémantique du modèle, certaines règles métier ne peuvent être exprimées que par des contraintes textuelles (ex : s'assurer qu'un solde est positif, ou marquer un sous-ensemble `{subset}`).

8.3 Paramètres

— `rule` (String) : La règle à afficher. Les accolades sont ajoutées automatiquement par le générateur.

8.4 Exemple

```
public class CompteBancaire {

    @UMLConstraint(rule = "solde >= decouverAutorise")
    private double solde; // Affichera {solde >= decouverAutorise} a
   cote de l'attribut
}
```

9 Les Liens Transversaux : `@UMLDependency`

9.1 Principe

Déclare une dépendance vers une autre classe qui n'est pas présente en tant qu'attribut de classe (ex : utilisée localement dans une méthode).

9.2 Pourquoi c'est utile ?

Cette annotation est la seule façon de révéler formellement sur le diagramme qu'une classe en utilise activement une autre, sans pour autant stocker son état de manière prolongée.

9.3 Paramètres

— `targetClass` (Class) : Classe ciblée.
— `label` (String) : Raison du lien (ex : `"utilise"`).

9.4 Exemple

```
@UMLDependency(targetClass = ServiceBancaire.class, label = "facture via
   ")
public class Panier {
    public void valider() {
        ServiceBancaire sb = new ServiceBancaire();
        sb.payer();
    }
}
```

10 La Documentation Visuelle : `@UMLNote`

10.1 Principe

Génère une bulle de commentaire (note jaune typique de l'UML) rattachée à une classe, un attribut ou une méthode.

10.2 Pourquoi c'est utile ?

Un diagramme explique le "Comment" (structure), la note explique le "Pourquoi". Elle est cruciale pour documenter un algorithme complexe ou avertir les autres développeurs d'une dette technique directement sur le schéma.

10.3 Paramètres

— `text` (String) : Contenu de la note.
— `position` (Enum) : `TOP`, `BOTTOM`, `LEFT`, `RIGHT`.

10.4 Exemple

```
public class MoteurDeRecherche {
    @UMLNote(text = "Algorithme critique, ne pas modifier", position =
    UMLPosition.RIGHT)
    public void indexer() {
    }
}
```

11 Le Regroupement Logique : `@UMLPackage`

11.1 Principe

Regroupe visuellement plusieurs classes au sein d'un espace nommé (un rectangle de *package* ou de *namespace* en PlantUML).

11.2 Pourquoi c'est utile ?

Sur un projet contenant 40 classes, le diagramme devient illisible. Regrouper les classes par domaine métier (ex : "Sécurité", "Facturation", "Catalogue") permet de diviser la complexité visuelle, sans forcer les développeurs à restructurer leurs vrais dossiers Java.

11.3 Paramètres

— `name` (String) : Nom du groupe.
— `color` (String) : Couleur de fond du groupe.

11.4 Exemple

```
@UMLPackage(name = "Module Paiement", color = "#EEEEEE")
@UMLClass
public class Facture {
}
```

12 Le Correcteur de Mise en Page : `@UMLLayout`

12.1 Principe

Crée un lien invisible (`-[hidden]->`) entre deux classes dans le fichier PlantUML final pour forcer le moteur de rendu à les positionner l'une par rapport à l'autre.

12.2 Pourquoi c'est utile ?

Le moteur PlantUML calcule les positions automatiquement. Parfois, il croise des flèches inutilement ou aligne mal les boites. Cette annotation permet de forcer PlantUML a dessiner comment on veut pouvoir visualiser certaines boites (ex : si on veut que la classe "Voiture" se situe juste au dessus de la classe "moteur" sur le graphique). Attention : C'est une annotation qui peut être dangereuse si utilisé trop souvent.

12.3 Paramètres

— `targetClass` (Class) : Classe cible du lien invisible.
— `direction` (Enum) : Ex : `DOWN` (force la cible à être en dessous).

12.4 Exemple

```
@UMLLayout(targetClass = BaseDeDonnees.class, direction = UMLDirection.
    DOWN)
@UMLClass
public class InterfaceUtilisateur {
}
```